

## CENTRO DE FORMAÇÃO PROFISSIONAL

### Relatório de Laboratório — Gestão de Utilizadores em PHP

Desenvolvimento de um sistema de registo, autenticação e gestão de utilizadores em PHP e MySQL, com melhorias de segurança sobre a base fornecida no enunciado

|                         |                                                                                                                                                                         |
|-------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Formando                | João Casanova Pinto (utilizador: bigamist)                                                                                                                              |
| Data                    | 28 de julho de 2026                                                                                                                                                     |
| Sistema operativo       | Ubuntu Server 26.04 LTS                                                                                                                                                 |
| Endereço IP do servidor | 192.168.1.93/24                                                                                                                                                         |
| Projeto                 | Gestão de Utilizadores, instalado em /var/www/html/gestao_utilizadores                                                                                                  |
| Base de dados           | gestao_utilizadores (utilizador dedicado gu_user)                                                                                                                       |
| Linguagens              | PHP (mysqli, prepared statements), MySQL, HTML, CSS                                                                                                                     |
| Continuação de          | Relatórios de Laboratório UC0635 (LAMP + Webmin), Webmin + Samba, Webmin + FTP, Webmin + SSH + phpMyAdmin, Utilizador admin + MySQL Workbench e Instalação do WordPress |

## 1. Objetivos

Na continuação da mesma VM usada nos laboratórios anteriores, este exercício teve como base o enunciado "Gestão de Utilizadores" fornecido pelo formador — uma aplicação PHP para registo, visualização, alteração e eliminação de utilizadores, com autenticação por nome de utilizador e palavra-passe. O enunciado incluía ainda uma lista de melhorias de segurança a implementar sobre o código-base fornecido. Este relatório documenta:

- A criação de uma pasta isolada para o projeto, sem interferir com os restantes trabalhos já alojados na mesma VM (WordPress, partilha FTP);
- A criação de uma base de dados e um utilizador MySQL dedicados, seguindo o mesmo padrão de segregação de acessos já usado no laboratório do WordPress;
- A implementação de todas as melhorias de segurança sugeridas no enunciado: sessões, logout, hash de palavra-passe, prepared statements, não exposição de dados sensíveis por GET, ficheiro único de credenciais, e bloqueio de conta após tentativas falhadas;
- A estilização final da aplicação.

## 2. Estrutura do Projeto e Base de Dados

### 2.1 Criação da pasta isolada do projeto

Através do PuTTY, foi criada uma pasta dedicada dentro da raiz do Apache, separada dos restantes projetos (wordpress, partilha FTP), garantindo que nenhum deles seria afetado:

```
$ sudo mkdir -p /var/www/html/gestao_utilizadores
$ sudo chown -R bigamist:bigamist /var/www/html/gestao_utilizadores
$ sudo chmod -R 755 /var/www/html/gestao_utilizadores
```

```

*** System restart required ***
Last login: Wed Jul 22 11:06:49 2026 from 192.168.1.67
bigamist@ubuntu-server:~$ sudo mkdir -p /var/www/html/gestao_utilizadores
(sudo: authenticate) Password:
bigamist@ubuntu-server:~$ sudo chown -R bigamist:bigamist /var/www/html/gestao_utilizadores
bigamist@ubuntu-server:~$ sudo chmod -R 755 /var/www/html/gestao_utilizadores
bigamist@ubuntu-server:~$ ls -la /var/www/html/
total 36
drwxr-xr-x 5 root      root      4096 Jul 28 11:21 .
drwxr-xr-x 3 root      root      4096 Jul 9 11:57 ..
drwxr-xr-x 4 bigamist  root      4096 Jul 17 11:42 bigamist
drwxr-xr-x 2 bigamist bigamist  4096 Jul 28 11:21 gestao_utilizadores
-rw-r--r-- 1 root      root      10672 Jul 9 11:57 index.html
-rw-r--r-- 1 root      root      20 Jul 9 12:27 teste.php
drwxr-xr-x 5 www-data www-data  4096 Jul 22 11:31 wordpress
bigamist@ubuntu-server:~$ cd /var/www/html/gestao_utilizadores
bigamist@ubuntu-server:/var/www/html/gestao_utilizadores$ touch acesso_bd.php log
in.php processar_login.php menu.php registrar_utilizador.php processar_registo.ph
p ver_utilizador.php alterar_utilizador.php processar_alterar.php eliminar_utili
zador.php processar_eliminar.php logout.php
ls -la
total 8
drwxr-xr-x 2 bigamist bigamist 4096 Jul 28 11:23 .
drwxr-xr-x 5 root      root      4096 Jul 28 11:21 ..
-rw-rw-r-- 1 bigamist bigamist 0 Jul 28 11:23 acesso_bd.php
-rw-rw-r-- 1 bigamist bigamist 0 Jul 28 11:23 alterar_utilizador.php
-rw-rw-r-- 1 bigamist bigamist 0 Jul 28 11:23 eliminar_utilizador.php
-rw-rw-r-- 1 bigamist bigamist 0 Jul 28 11:23 login.php
-rw-rw-r-- 1 bigamist bigamist 0 Jul 28 11:23 logout.php
-rw-rw-r-- 1 bigamist bigamist 0 Jul 28 11:23 menu.php
-rw-rw-r-- 1 bigamist bigamist 0 Jul 28 11:23 processar_alterar.php
-rw-rw-r-- 1 bigamist bigamist 0 Jul 28 11:23 processar_eliminar.php
-rw-rw-r-- 1 bigamist bigamist 0 Jul 28 11:23 processar_login.php
-rw-rw-r-- 1 bigamist bigamist 0 Jul 28 11:23 processar_registo.php
-rw-rw-r-- 1 bigamist bigamist 0 Jul 28 11:23 registrar_utilizador.php
-rw-rw-r-- 1 bigamist bigamist 0 Jul 28 11:23 ver_utilizador.php
bigamist@ubuntu-server:/var/www/html/gestao_utilizadores$

```

Figura 1 — Pasta gestao\_utilizadores criada de forma isolada em /var/www/html/, ao lado dos projetos wordpress e bigamist (FTP), com os 12 ficheiros PHP do projeto já criados.

## 2.2 Recuperação do acesso ao MySQL

Ao tentar aceder ao MySQL para criar a base de dados do projeto, verificou-se que a password do utilizador root era desconhecida — uma consequência da alteração do método de autenticação (de auth\_socket para caching\_sha2\_password) realizada no laboratório anterior do phpMyAdmin, cuja password não tinha ficado registada.

O acesso foi recuperado através do arranque do MySQL em modo de recuperação, sem verificação de privilégios:

```

$ sudo mkdir -p /var/run/mysqld
$ sudo chown mysql:mysql /var/run/mysqld
$ sudo mysqld_safe --skip-grant-tables --skip-networking &

```

Já dentro dessa sessão sem password, a password do root foi reposta, e o serviço normal do MySQL foi restaurado:

```

mysql> FLUSH PRIVILEGES;
mysql> ALTER USER 'root'@'localhost' IDENTIFIED WITH caching_sha2_password BY
'.....';
mysql> FLUSH PRIVILEGES;
$ sudo pkill mysqld
$ sudo rm -f /var/run/mysqld/mysqld.sock /var/run/mysqld/mysqld.sock.lock
$ sudo systemctl start mysql

```

```

Copyright (c) 2000, 2026, Oracle and/or its affiliates.

Oracle is a registered trademark of Oracle Corporation and/or its
affiliates. Other names may be trademarks of their respective
owners.

Type 'help;' or '\h' for help. Type '\c' to clear the current input statement.

mysql> USE gestao_utilizadores;
Reading table information for completion of table and column names
You can turn off this feature to get a quicker startup with -A

Database changed
mysql> INSERT INTO utilizadores (nome_utilizador, palavra_passe, email)
-> VALUES ('admin', '$2y$12$nkEa6mxd4cbA0G/17LjpL08IdeINLlElknT6TMrdoU70Qhr4Phl9m', 'admin@teste.com
');
Query OK, 1 row affected (0.00 sec)

mysql>

```

Figura 2 — Sessão MySQL já recuperada: seleção da base de dados e inserção do primeiro utilizador de teste (admin), com a palavra-passe já em hash.

## 2.3 Criação da base de dados e do utilizador dedicado gu\_user

Seguindo o mesmo padrão de segregação de acessos já usado no laboratório do WordPress (utilizador dedicado wpuser), foi criado um utilizador MySQL específico para esta aplicação, gu\_user, com privilégios restritos apenas à base de dados gestao\_utilizadores — nunca ligando a aplicação com a conta root:

```
CREATE DATABASE gestao_utilizadores;
USE gestao_utilizadores;

CREATE TABLE utilizadores (
    id_utilizador INT NOT NULL AUTO_INCREMENT,
    nome_utilizador VARCHAR(30) NOT NULL,
    palavra_passe VARCHAR(255) NOT NULL,
    email VARCHAR(75) NOT NULL,
    PRIMARY KEY (id_utilizador),
    UNIQUE KEY (nome_utilizador)
);

CREATE USER 'gu_user'@'localhost' IDENTIFIED BY '.....';
GRANT ALL PRIVILEGES ON gestao_utilizadores.* TO 'gu_user'@'localhost';
FLUSH PRIVILEGES;
```

*Nota: o campo palavra\_passe foi definido com VARCHAR(255) em vez do VARCHAR(8) do enunciado original — necessário porque um hash gerado por password\_hash() ocupa cerca de 60 caracteres, e ficaria cortado (tornando o login impossível) com o tamanho original.*

## 2.4 Ficheiro único de credenciais (acesso\_bd.php)

Conforme sugerido no enunciado, todas as credenciais de ligação à base de dados foram centralizadas num único ficheiro, incluído em todas as páginas que necessitam de aceder à base de dados:

```
<?php
$servidor = "localhost";
$utilizador_bd = "gu_user";
$password_bd = ".....";
$base_dados = "gestao_utilizadores";

$ligacao = mysqli_connect($servidor, $utilizador_bd, $password_bd,
$base_dados);

if (!$ligacao) {
    die("Falha na ligação à base de dados: " . mysqli_connect_error());
}
?>
```

● **Resultado:** Pasta do projeto isolada, base de dados gestao\_utilizadores e utilizador dedicado gu\_user criados com sucesso, com privilégios restritos a essa base de dados. Ligação centralizada num único ficheiro acesso\_bd.php, testada e validada.

## 3. Autenticação, Sessões e Logout

O código de autenticação fornecido no enunciado (processar\_login.php) continha uma vulnerabilidade de injeção SQL, ao construir a query por concatenação direta dos valores recebidos do formulário. Esta foi substituída por uma prepared statement, e a palavra-passe passou a ser validada por hash (password\_hash / password\_verify) em vez de comparação direta em texto simples:

```
$sql = "SELECT id_utilizador, nome_utilizador, palavra_passe FROM utilizadores
WHERE nome_utilizador = ?";
$stmt = mysqli_prepare($ligacao, $sql);
```

```

mysqli_stmt_bind_param($stmt, "s", $username);
mysqli_stmt_execute($stmt);
$resultado = mysqli_stmt_get_result($stmt);

if (mysqli_num_rows($resultado) == 1) {
    $utilizador = mysqli_fetch_assoc($resultado);
    if (password_verify($password, $utilizador['palavra_passe'])) {
        $_SESSION['utilizador_id'] = $utilizador['id_utilizador'];
        header("Location: menu.php");
        exit;
    }
}
}

```

Note-se ainda que a mensagem de erro é sempre igual ("Erro de autenticação!"), quer o nome de utilizador não exista, quer a palavra-passe esteja errada — uma boa prática que evita que um atacante descubra, por tentativa e erro, quais os nomes de utilizador válidos no sistema.

Figura 3 — Formulário de autenticação (*login.php*), já ligado ao *processar\_login.php* com sessões.

Figura 4 — Redireção bem-sucedida para *menu.php* após autenticação válida (página em branco nesta fase, por o ficheiro ainda não ter conteúdo).

O *menu.php* foi implementado com verificação de sessão (nesta fase, ainda com o bloco repetido em cada página — centralizado posteriormente na secção 8) e um link de Logout:

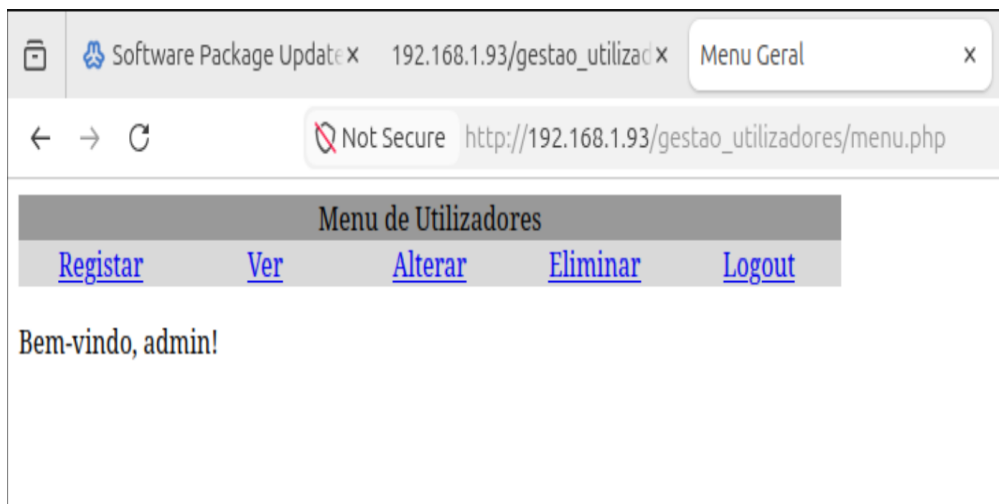


Figura 5 — Menu de utilizadores apresentado após autenticação bem-sucedida, com a mensagem de boas-vindas e o link de Logout.

O logout.php destrói a sessão ativa e redireciona sempre para o login. Foi confirmado que, após o logout, uma tentativa de acesso direto ao menu.php (sem nova autenticação) é corretamente bloqueada e redirecionada para o login:

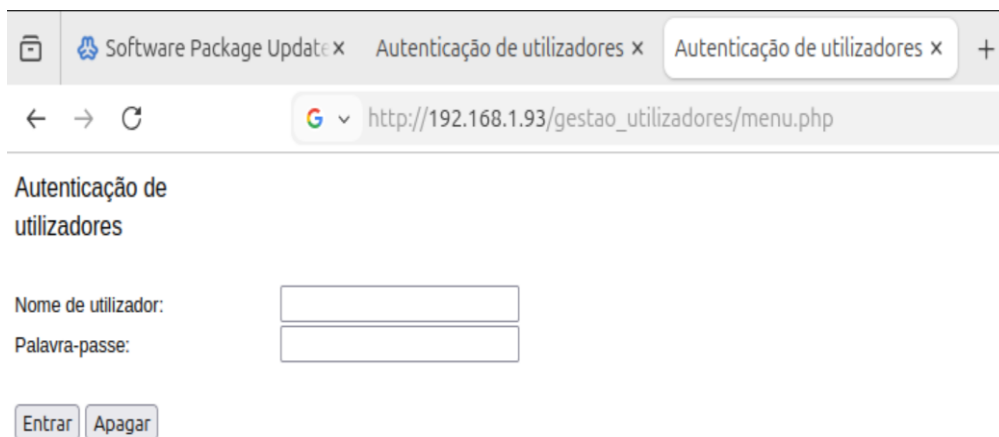


Figura 6 — Tentativa de acesso direto a menu.php após logout, corretamente redirecionada para login.php, confirmando a proteção de sessão.

● **Resultado:** Login, sessões e logout implementados e testados com sucesso: autenticação com palavra-passe em hash, prepared statements sem SQL injection, e proteção de sessão a bloquear o acesso direto às páginas internas sem autenticação prévia.

#### 4. Registo de Utilizadores

O formulário e o processamento do registo (registar\_utilizador.php / processar\_registo.php) foram implementados com o mesmo cuidado: validação dos campos obrigatórios, hash da palavra-passe com password\_hash() antes de gravar na base de dados, e inserção por prepared statement:

```
$password_hash = password_hash($_POST['password'], PASSWORD_DEFAULT);

$sql = "INSERT INTO utilizadores (nome_utilizador, palavra_passe, email)
VALUES (?, ?, ?)";
$stmt = mysqli_prepare($ligacao, $sql);
mysqli_stmt_bind_param($stmt, "sss", $username, $password_hash, $email);
mysqli_stmt_execute($stmt);
```

Figura 7 — Formulário de registo de utilizadores, acedido a partir do menu, já com sessão ativa.

Figura 8 — Confirmação de registo bem-sucedido do novo utilizador pedro.

- **Resultado:** Registo de novos utilizadores implementado e testado com sucesso, com palavra-passe gravada apenas em hash na base de dados.

## 5. Visualização de Utilizadores

O `ver_utilizador.php` lista todos os utilizadores registados, ordenados por nome, usando `htmlspecialchars()` em todos os valores apresentados para evitar problemas de XSS:



Figura 9 — Lista de utilizadores registados (admin e pedro), com número de registo, nome e email.

- **Resultado:** Listagem de utilizadores implementada e validada, apresentando corretamente os registos existentes na base de dados.

## 6. Alteração de Dados do Utilizador

Ao contrário do enunciado original — que passava o nome de utilizador e o email na própria barra de endereço (?id\_utilizador=...&nome\_utilizador=...&email=...) — a versão implementada passa apenas o número de registo (?id=), evitando expor dados pessoais na URL. Os restantes dados são obtidos diretamente da base de dados, com base nesse id:



Figura 10 — Lista de utilizadores para seleção do registo a alterar, com o link contendo apenas o id na URL.

Software Package Update x Autenticação de utilizadores x Autenticação de utilizadores x Confirmação de alteração

← → ↻ Not Secure http://192.168.1.93/gestao\_utilizadores/processar\_alterar.php?id=1

Menu de Utilizadores

[Registar](#) [Ver](#) [Alterar](#) [Eliminar](#) [Logout](#)

Bem-vindo, admin!

Nº registo: 1

Nome de utilizador: admin

Endereço de correio eletrónico: admin@teste.com

Dados do utilizador a editar:

Alterar dados

Figura 11 — Formulário de alteração de dados, pré-preenchido com os valores atuais do utilizador selecionado (URL: `processar_alterar.php?id=1`).

Numa primeira versão do formulário de edição, os campos de texto (nome e email) ficaram, por lapso, fora da tag `<form>` — apenas o botão e os campos ocultos estavam dentro dela. Como resultado, ao submeter o formulário, os valores editados no ecrã não eram enviados, e a alteração aparentava não ter efeito:

Software Package Update x Autenticação de utilizadores x Autenticação de utilizadores x Menu Geral

← → ↻ Not Secure http://192.168.1.93/gestao\_utilizadores/processar\_alterar.php

Menu de Utilizadores

[Registar](#) [Ver](#) [Alterar](#) [Eliminar](#) [Logout](#)

Bem-vindo, admin!

Alterado com sucesso! [Clique para continuar](#)

Figura 12 — Mensagem "Alterado com sucesso!" apesar do email não ter sido de facto atualizado (bug detetado nesta fase).



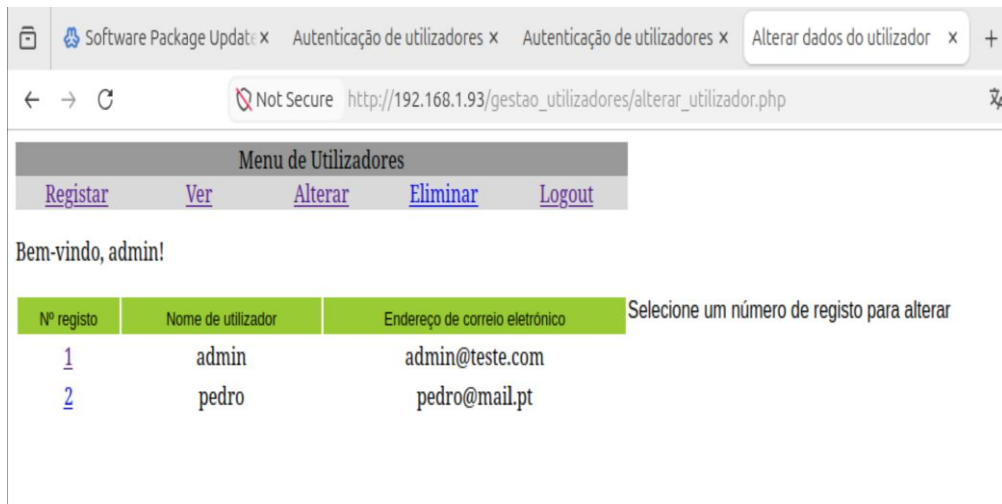


Figura 13 — Confirmação do problema: o email do utilizador admin permanece inalterado, apesar da mensagem de sucesso.

O problema foi corrigido movendo a tag `<form>` para envolver a totalidade da tabela (campos de texto e botão), garantindo que todos os valores editados são efetivamente submetidos:

```
<form method="POST" action="processar_alterar.php">
<table ...>
  ... campos de texto (nome_user, email_user) ...
  <input type="submit" name="alterar" value="Alterar dados">
  <input type="hidden" name="id_utilizador" value="...">
</table>
</form>
```

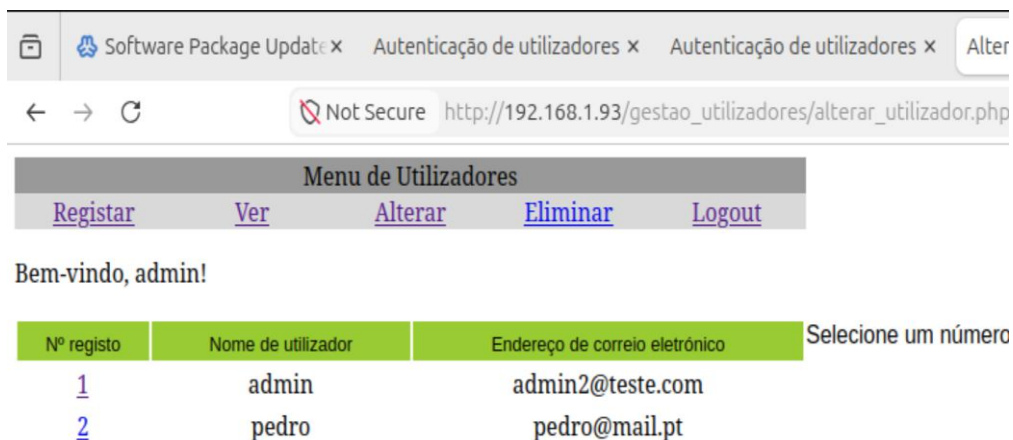


Figura 14 — Após a correção, o email do utilizador admin foi corretamente atualizado para `admin2@teste.com`.

- **Resultado:** Alteração de dados implementada, com um erro real de estrutura de formulário identificado e corrigido durante os testes, e validada com sucesso após a correção.

## 7. Eliminação de Utilizadores

Seguindo o mesmo padrão do fluxo de Alterar (apenas o id na URL, confirmação antes de executar a ação destrutiva), foi implementado o fluxo de eliminação de utilizadores. Para o teste, foi registado um utilizador adicional (joao), especificamente para ser eliminado, preservando os utilizadores admin e pedro:



Figura 15 — Lista de utilizadores para eliminação, incluindo o utilizador de teste joao (registo nº 3).

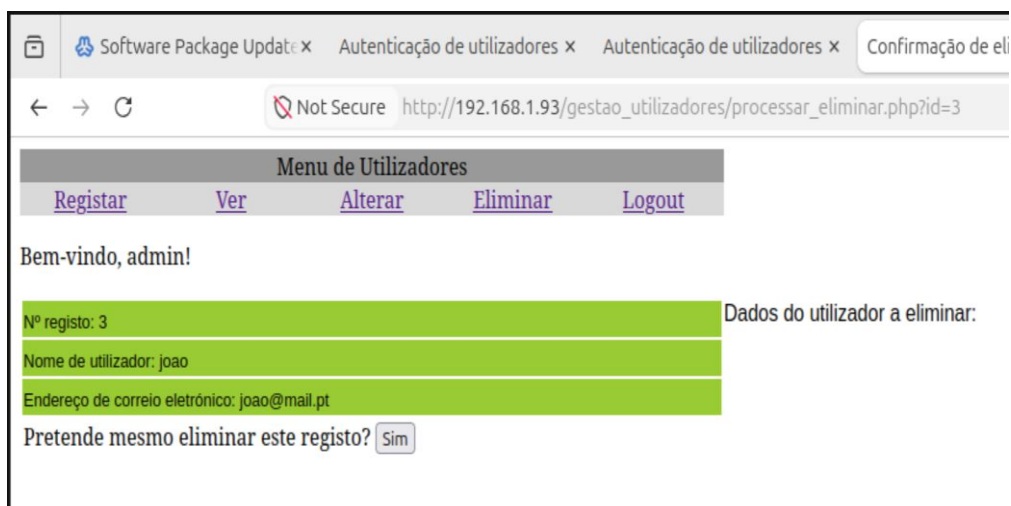


Figura 16 — Ecrã de confirmação antes da eliminação, apresentando os dados do utilizador joao e pedindo confirmação explícita.



Figura 17 — Confirmação de eliminação bem-sucedida.

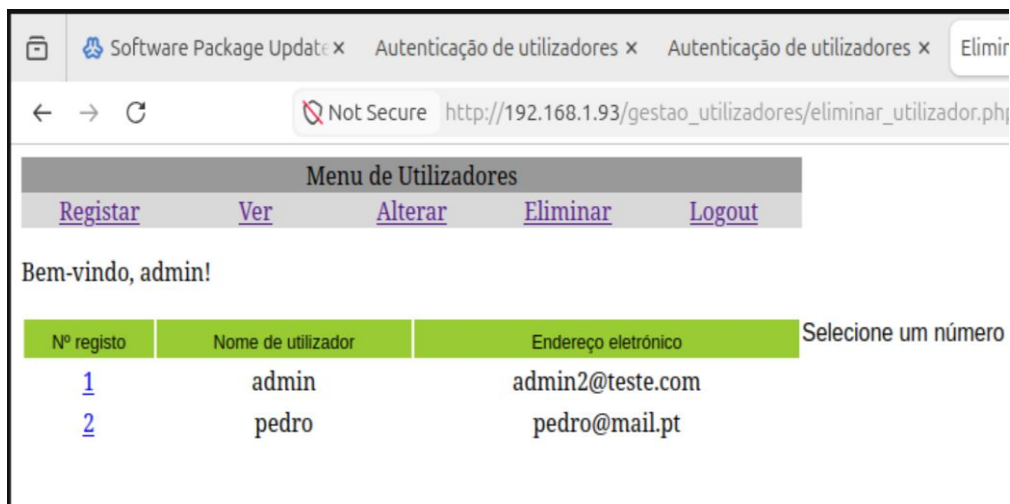


Figura 18 — Lista final, confirmando que o utilizador joao foi removido e que admin e pedro permanecem intactos.

- **Resultado:** Eliminação de utilizadores implementada e testada com sucesso, incluindo o ecrã de confirmação explícita antes da remoção definitiva do registo.

## 8. Centralização da Verificação de Sessão

Conforme sugerido no enunciado, o bloco de verificação de sessão (repetido no topo de cada página protegida) foi centralizado num único ficheiro, `verificar_sessao.php`, incluído por todas as páginas que exigem autenticação:

```
<?php
session_start();
if (!isset($_SESSION['utilizador_id'])) {
    header("Location: login.php");
    exit;
}
?>
```

Cada página passou a incluir apenas uma linha (`include 'verificar_sessao.php';`) em vez de repetir as quatro linhas do bloco de sessão, reduzindo duplicação de código e centralizando a lógica de proteção de acesso.



Figura 19 — Menu apresentado corretamente após login, já com a verificação de sessão centralizada em todas as páginas.

● **Resultado:** Verificação de sessão centralizada com sucesso num único ficheiro (verificar\_sessao.php), com o fluxo completo (login, registo, listagem, alteração, eliminação, logout, proteção de acesso) reconfirmado a funcionar sem regressões.

## 9. Bloqueio de Conta Após Tentativas Falhadas

Foi implementado o bloqueio automático de conta ao fim de 3 tentativas de autenticação falhadas, conforme pedido no enunciado, com bloqueio permanente até desbloqueio manual (via SQL). A tabela utilizadores foi alterada com duas colunas novas:

```
ALTER TABLE utilizadores ADD COLUMN tentativas_falhadas INT NOT NULL DEFAULT 0;  
ALTER TABLE utilizadores ADD COLUMN bloqueado TINYINT(1) NOT NULL DEFAULT 0;
```

No processar\_login.php, cada tentativa falhada incrementa o contador; ao atingir 3, a conta é marcada como bloqueada e passa a recusar o acesso mesmo com a palavra-passe correta. Um login bem-sucedido repõe o contador a zero:

```
if ($utilizador['bloqueado'] == 1) {  
    echo "Conta bloqueada devido a demasiadas tentativas falhadas. Contacte o administrador.";  
    exit;  
}  
  
// password errada:  
$novas_tentativas = $utilizador['tentativas_falhadas'] + 1;  
if ($novas_tentativas >= 3) {  
    // UPDATE ... SET tentativas_falhadas = ?, bloqueado = 1 ...  
}
```



Figura 20 — Conta do utilizador pedro bloqueada após a 3ª tentativa de autenticação falhada.



Figura 21 — Confirmação de que o bloqueio persiste mesmo ao introduzir a palavra-passe correta, comprovando que o bloqueio é independente das credenciais.

- **Resultado:** Bloqueio automático de conta após 3 tentativas falhadas implementado e validado, incluindo a confirmação de que o bloqueio impede o acesso mesmo com credenciais corretas.

## 10. Estilização (CSS)

Numa fase final, após validar toda a funcionalidade, foi aplicada uma folha de estilos (estilo.css) a todas as páginas, usando a paleta (Jade #33CC99, Jade Profundo #1E9E76, Tinta #24262B, Papel #FAFAF8, Jade Claro #E3F9F0; tipografia Space Grotesk para títulos e Inter para o corpo de texto):



Figura 22 — Formulário de login já estilizado.



Figura 23 — Menu de utilizadores estilizado, com a barra em Jade e texto em Tinta.



Figura 24 — Formulário de registo estilizado.



Figura 25 — Listagem de utilizadores estilizada, com cabeçalhos em Jade Claro.



Figura 26 — Ecrã de seleção para alteração, estilizado.



Figura 27 — Ecrã de seleção para eliminação, estilizado.

- **Resultado:** Estilização aplicada com sucesso a todas as páginas do projeto, sem qualquer perda de funcionalidade.

## 11. Observações e Aprendizagens

- O root do MySQL pode ficar inacessível após uma mudança de método de autenticação (auth\_socket → caching\_sha2\_password) feita num laboratório anterior sem a password ter sido registada; o arranque em modo --skip-grant-tables permite recuperar o acesso sem perder dados, seguido de um ALTER USER para repor uma password conhecida;
- A criação de um utilizador MySQL dedicado a cada aplicação (gu\_user, à semelhança do wpuser do WordPress), com privilégios restritos apenas à base de dados dessa aplicação, é uma boa prática recorrente para limitar o impacto de uma eventual falha de segurança;
- Um campo dimensionado para o valor original de uma palavra-passe em texto simples (VARCHAR(8), como no enunciado) é demasiado pequeno para armazenar um hash gerado por password\_hash() (cerca de 60 caracteres) — é necessário redimensionar a coluna antes de introduzir hashing;
- Um formulário HTML só envia os campos que estão fisicamente dentro da tag <form> — campos de texto colocados fora dela (mesmo que visualmente pareçam parte do mesmo formulário) nunca chegam ao servidor, resultando num "sucesso" enganador em que a operação parece ter corrido bem mas não altera nada;

- Passar apenas um identificador numérico (id) na barra de endereço, em vez de todos os dados do registo (nome, email), evita expor informação pessoal na URL, sem complicar a implementação — os restantes dados são sempre obtidos da base de dados a partir desse id;
- Centralizar lógica repetida (como a verificação de sessão) num único ficheiro incluído por todas as páginas reduz a duplicação de código e facilita alterações futuras, sem qualquer impacto funcional quando bem testado;
- Um bloqueio de conta após tentativas falhadas só é eficaz se persistir mesmo perante credenciais corretas subsequentes — caso contrário, um atacante poderia contornar o bloqueio bastando acertar a palavra-passe a tempo.

João Pinto

28/07/2026